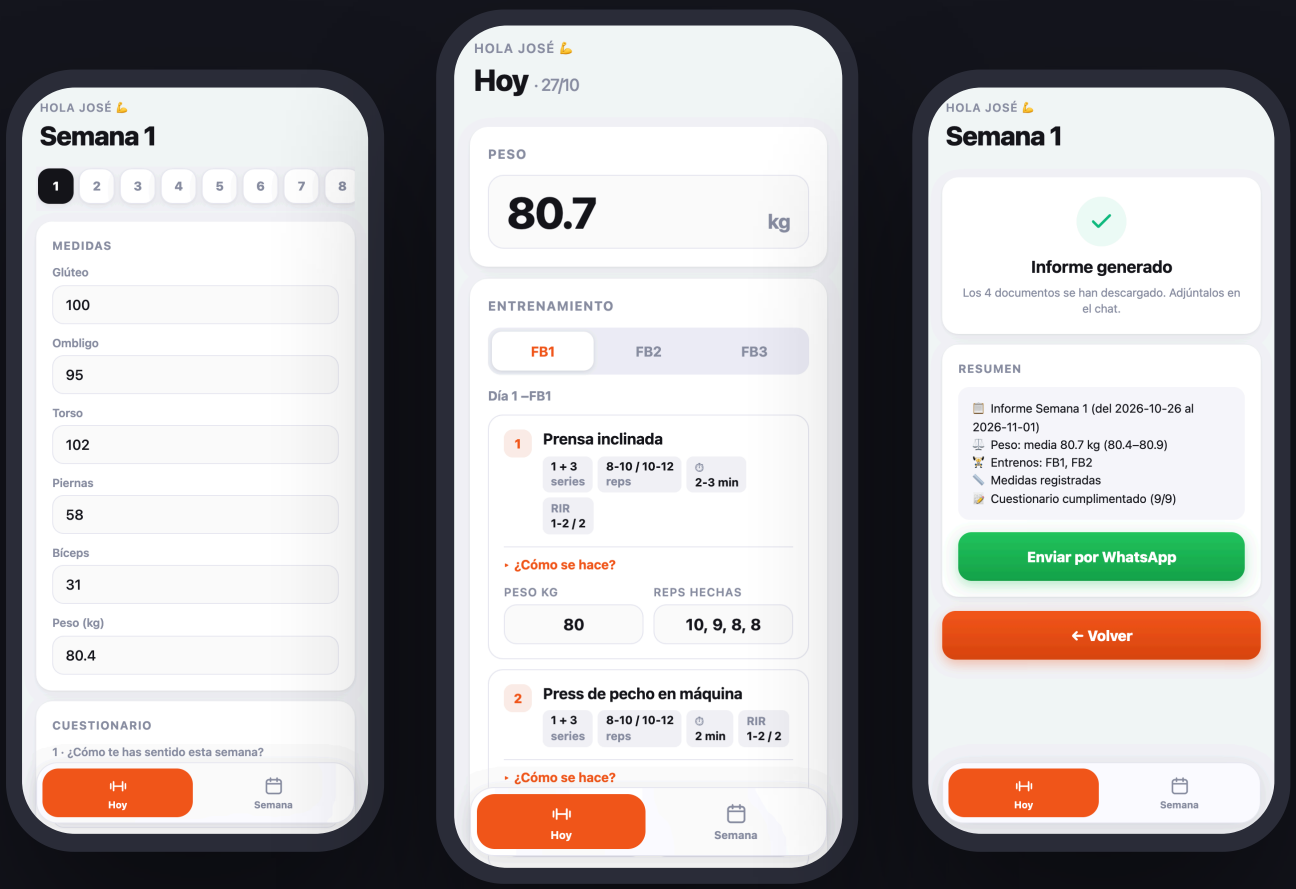
GUÍA PASO A PASO

Jose Mas · lumicatech.es

Tu app de gimnasio con IA, desde cero y gratis

Cómo construí —con un agente de IA— la app que registra mi entrenamiento y genera los partes semanales de mi entrenador personal.

TIEMPO	COSTE HOSTING	PROGRAMAR
2–3 horas	0 €	No hace falta



0 Qué vas a construir

Una web que funciona como app en el móvil con la que:

CADA DÍA

Registras tu peso y el entrenamiento del día: eliges la sesión (FB1/FB2/FB3), ves las **series y reps pedidas** por tu entrenador junto a cada ejercicio, y apuntas el peso y las reps que has hecho.

CADA SEMANA

Rellenas medidas corporales y un cuestionario de 9 preguntas. Pulsas **Generar informe** y la app produce los 4 documentos que espera tu entrenador (2 Excel + 1 Excel + 1 Word), con su formato original, y prepara el mensaje de WhatsApp.

SIEMPRE

Todo se guarda en tu navegador. Sin servidores, sin base de datos, sin cuotas. Publicada gratis en GitHub Pages.

> **Requisitos previos:** un Mac (o PC con Linux/Windows), conexión a internet, una cuenta gratuita en GitHub y ganas de 2–3 horas. No se necesita saber programar. **Coste:** 0 € en hosting y librerías. La única cuenta de pago posible es la del asistente de IA (ver paso 2).

1 Prepara tu material

Reúne en una carpeta los documentos que hoy rellenas a mano para tu entrenador (en mi caso: control de peso semanal, seguimiento de medidas, plan de entrenamiento en Excel y cuestionario en Word). Estos archivos serán las **plantillas**: la app los leerá y los rellenará con tu formato exacto.

Esto era exactamente lo que llevaba cada documento mío, y lo que la app aprende a leer y rellenar:

Documento	Qué contiene	Qué extrae y rellena la app
Control de peso semanal (.xlsx)	Una fila por semana, una columna por día (lunes a domingo) y una columna de promedio con fórmula	Mi peso corporal de cada mañana, en la fila de la semana en curso
Seguimiento de peso y medidas (.xlsx)	Una fila por medida (glúteo, ombligo, torso, piernas, bíceps, peso) y una columna por semana	Las 6 medidas, en la columna de la semana que toca
Plan de entrenamiento (.xlsx)	3 bloques (Día 1/2/3) con ejercicios y sus series, reps, descanso y RIR pedidos, y una columna vacía "Peso utilizado"	El peso y las reps reales de cada ejercicio, en la columna de su fila. Las series pedidas se leen de aquí para mostrarlas en la app
Cuestionario semanal (.docx)	9 preguntas: sensaciones, hambre, saciedad, dieta, plan, sueño, pasos/cardio, motivación y sugerencias	Mis 9 respuestas, bajo cada pregunta, manteniendo el formato original

> **La IA no adivina:** necesita documentos con una estructura clara y que se la describas. Si tus documentos son diferentes a los míos, no pasa nada — enséñaselos al asistente y dile qué dato va en cada sitio (consejo: empieza con un solo documento, verifica que lo rellena bien, y suma el siguiente). Si metes algo sin estructura (notas sueltas, capturas...), los resultados serán imprevisibles.

La carpeta puedes crearla como cualquier otra (en el Finder: nueva carpeta "personal-trainer" en tu carpeta personal) o con el terminal:

```
TERMINAL
mkdir -p ~/personal-trainer
# copia dentro tus .xlsx y .docx tal cual te los da tu entrenador
```

2 Instala las herramientas (una sola vez)

> **Necesitas:** un Mac o PC, cuenta gratuita en github.com y cuenta de claude.ai. El resto es gratis.

2.1 Aprende a abrir el terminal

El terminal es una ventana donde escribes comandos. En Mac: `Cmd` + `Espacio`, escribe "Terminal" y pulsa `Enter`. Todo lo que aparece en las cajas oscuras de esta guía se escribe ahí, línea a línea.

2.2 Node.js (el motor de la instalación del asistente)

Descarga el instalador LTS de nodejs.org, dale a siguiente-siguiente-terminar y verifica que quedó bien instalado escribiendo en el terminal:

```
TERMINAL
node --version      # debe responder algo como v22.x.x (es solo una comprobación)
```

Node trae de regalo `npm`, el instalador de herramientas que usaremos a continuación.

2.3 Visual Studio Code (el IDE)

Descarga VS Code (code.visualstudio.com) y arrástralo a Aplicaciones. Es donde verás el código que genera la IA y donde puedes retocarlo.

Cuando llegue el momento, la forma de "abrir el proyecto" es: abre VS Code → menú **Archivo → Abrir carpeta...** → elige `personal-trainer`. Verás tus plantillas a la izquierda y, a partir de ahora, todo el código que cree la IA.

2.4 Claude Code (el asistente)

Claude Code es un agente de programación que trabaja dentro de tu proyecto desde el terminal: lee tus archivos, escribe el código, ejecuta tests y te va contando lo que hace. Instálalo con este comando (el primero puede tardar un minuto):

```
TERMINAL
npm install -g @anthropic-ai/claude-code
```

Cómo se abre en tu proyecto (el flujo de siempre, del paso 6 en adelante):

1. Abre VS Code y la carpeta del proyecto (**Archivo → Abrir carpeta...**).
2. Menú **Terminal → Nuevo terminal**: se abre una franja abajo. Ya está dentro de tu carpeta, sin comandos raros.
3. Escribe `claude` y pulsa `Enter`.

```
TERMINAL
claude
```

La primera vez te pedirá iniciar sesión: se abre el navegador, entras en tu cuenta y vuelves a VS Code, listo para escribirle prompts en lenguaje normal. Todos los días igual: VS Code, carpeta, terminal, `claude`. Hay plan gratuito con límites.

3 SESIÓN 1

Analiza tus plantillas y crea el prototipo

Con Claude Code abierto en tu carpeta, pega este prompt (adapta los nombres de tus archivos):

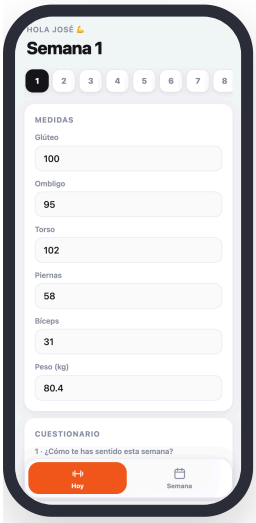
PROMPT PARA CLAUDE CODE

```
Tengo estos documentos que relleno cada semana para mi entrenador personal:
[control_de_peso.xlsx, medidas.xlsx, entrenamiento.xlsx, cuestionario.docx]. Es un rollo ir
entre Drive, el ordenador y WhatsApp.

Quiero una app donde registre mi peso diario y mi entrenamiento, y que al final de la semana
genere esos 4 documentos rellenados para enviarlos por WhatsApp a mi entrenador.

Empieza analizando la estructura de las plantillas y propón el plan.
```

El agente inspeccionará las celdas exactas de cada Excel, la estructura del Word, y te propondrá la arquitectura. En mi caso eligió: aplicación Python con Flask que guarda los datos en un JSON y rellena copias de las plantillas con openpyxl y python-docx. Deja que trabaje y luego prueba tú: abre la URL que te indique (algo como `http://localhost:5000`) en el navegador.



> **Qué comprobar en esta fase:** que puedas registrar el peso del día, rellenar medidas y cuestionario, y que el botón de generar informe produzca 4 archivos correctos. Si algo no cuadra con tus plantillas, díselo: *"el peso de la semana debe ir en la fila X, columna Y"*. La IA corrige sobre la marcha.

4 SESIÓN 2

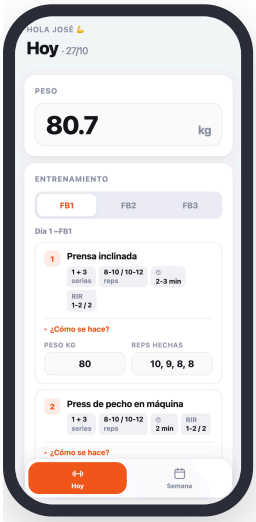
Diseño de app de verdad y registro por días

El primer prototipo funciona pero es feo y no se ajusta a cómo entrenas. Segunda tanda de prompts:

PROMPT PARA CLAUDE CODE

```
Ahora vamos a cuidar el diseño, optimizado para móvil, con calidad de app profesional. Revisa los estándares de apps de fitness del mercado (Hevy, Strong): barra de pestañas inferior, controles segmentados, tipografía 16px+ para evitar el zoom de iOS, safe-areas.

Además el entrenamiento debe ser editable por día: no hago todos los ejercicios el mismo día. Lee los días (Día 1, Día 2, Día 3) del propio Excel del plan y muestra solo los ejercicios de cada día, con las series y reps pedidas al lado, y campos para el peso y las reps que he hecho en cada serie.
```



Resultado: cabecera estilo iOS, selector segmentado FB1/FB2/FB3, tarjetas por ejercicio con las series pedidas (chips con series, reps, descanso y RIR) y campos de registro grandes para el pulgar.

El plan se **lee directamente del Excel del entrenador**, así que cuando cambie la rutina, la app se actualiza sola.

Consejo: revisa la app en el móvil tras cada cambio y pide ajustes concretos: *"no quiero ver los 3 días de entrenamiento juntos, solo el que toca hoy"*, *"los campos de reps deben aceptar varias series: 10, 9, 8"*. Cuanto más concreto el cambio, mejor el resultado; y cada iteración es más rápida que la anterior.

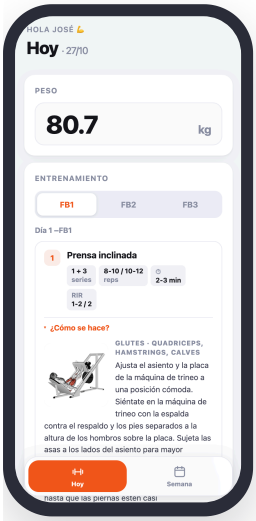
5

SESIÓN 3 · BONUS

Fichas de ejercicios con GIF

PROMPT PARA CLAUDE CODE

```
Integra un dataset abierto de ejercicios para que cada ejercicio tenga una ficha desplegable
"¿Cómo se hace?" con GIF animado e instrucciones en español. Usa el dataset de
github.com/hasaneyldrm/exercises-dataset (1.324 ejercicios, MIT) y mapea los nombres en español
de mi plan a los del catálogo.
```



GIF animado del movimiento, músculos trabajados e instrucciones paso a paso en español (el dataset trae 10 idiomas).

Solo hizo falta mapear los 19 nombres de mi rutina; el resto es automático.

Antes de publicar: qué significa que el repo sea público. Un repositorio público es exactamente eso: cualquiera puede ver su contenido sin registrarse, de forma permanente mientras no lo borres. Quedarán visibles el código (la gracia del post: que se vea cómo está hecha), las plantillas que subas —en mi caso el plan de mi entrenador, y decidí que no me importaba— y cualquier otro archivo de la carpeta. Revísala antes del `git push`. Tus registros, en cambio, no se publican: viven en el navegador de cada visitante.

6

SESIÓN 4

La convertimos en web estática y la publicamos gratis

6.0 Antes de empezar: GitHub CLI

Para publicar usaremos la herramienta `gh` de GitHub. Descarga su instalador en cli.github.com, instálala y autentícate (se abre el navegador: copia el código que te muestre el terminal y pégalo ahí):

TERMINAL

```
gh auth login
# elige: GitHub.com → HTTPS → "Login with a web browser"
```

Una app Python necesita un servidor. Para que viva gratis para siempre, se convierte a **web estática**: los datos se guardan en el navegador (localStorage) y los documentos se generan en el propio navegador con las librerías JavaScript ExcelJS y docx. Prompt:

PROMPT PARA CLAUDE CODE

```
Convierte la app a una web 100% estática para publicarla en GitHub Pages: mismos datos en
localStorage con export/import de copia de seguridad JSON, y generación de los 4 documentos en
el navegador con ExcelJS (xlsx) y docx (Word). Sustituye la lógica de Flask y deja la app lista
para subir.
```

Y para publicar (crea un repo público; tus *datos* nunca se suben, se quedan en tu navegador):

TERMINAL

```
git init && git add -A && git commit -m "mi app de gym"
gh repo create TUUSUARIO/mi-gym --public --source=. --push
gh api repos/TUUSUARIO/mi-gym/pages -X POST \
      -f "source[branch]=main" -f "source[path]=/"
```

Un minuto después: `https://tuusuario.github.io/mi-gym/`. En el iPhone: **Compartir → Añadir a pantalla de inicio** y la tienes como app.

> **Si falla:** si `gh repo create` responde que no estás autenticado, ejecuta `gh auth login` (paso 6.0) y vuelve a intentarlo. Y si el push falla con un error de "publickey", cambia el remoto a HTTPS: `git remote set-url origin https://github.com/TUUSUARIO/mi-gym.git`

7 Pruébalo todo antes de fiarte

- Registra un día de entrenamiento con 2-3 ejercicios y comprueba la tira semanal.
- Rellena medidas y cuestionario de la semana 1.
- Pulsa **Generar informe**: se descargan los 4 documentos. Ábrelos y compara celdas con las plantillas originales.
- Pulsa **Enviar por WhatsApp**: se abre el chat con el resumen. Adjunta los archivos.
- Exporta la copia de seguridad JSON y guárdala. Es tu "drive" ahora.

Pedirle al agente que escriba tests automáticos (*"añade pruebas con Node de la generación de documentos"*) es lo que te ahorra las sorpresas de un domingo a las 22:47.

8 Problemas típicos y salida

Síntoma	Solución
`git@github.com: Permission denied (publickey)`	Tu SSH no está configurado: usa el remoto HTTPS (`git remote set-url origin https://github.com/TUUSUARIO/mi-gym.git`), que autentica con el token de `gh`.
No cargo la web en el iPhone	Usa la URL de GitHub Pages (no localhost) o la IP local del ordenador en la misma red Wi-Fi.
Cambié de móvil y no están mis datos	Los datos viven por navegador: importa el JSON de copia de seguridad.
El entrenador cambió el plan de ejercicios	Actualiza el Excel plantilla y pídele a la IA que regenere los datos del plan; el resto es automático.

9 Costes, tiempo y lo que aprendí

	Antes	Ahora
Los datos	Por cada lado: en el Excel del móvil, en una nota, en la cabeza	Todo a mano, en el mismo sitio, desde el primer día
Rellenar	Abrir cada documento, editar celdas en Word y Excel, borrar lo de la semana anterior, releer para no equivocarte	Apuntar al momento y pulsar un botón: los documentos se generan solos con mis datos
Coste	0 € y frustración	0 € de hosting (Pages); la cuenta de IA es lo único de pago
Stack	—	HTML + CSS + JS, ExcelJS, docx, localStorage, GitHub Pages

La lección no es "sé programador". Es que describir bien un problema aburrido se ha convertido en una habilidad productiva: el agente escribió todo el código; yo aporté las plantillas y el criterio de diseño.

Importante: esto es un MVP personal, un proyecto de un fin de semana para un problema concreto (una persona, sus datos en su navegador). Cuando el problema es de verdad profesional —varios usuarios, base de datos, integraciones, facturación— el proyecto pide otro nivel: arquitectura, seguridad, mantenimiento. Para eso existe **lumicatech.es**, donde estaremos encantados de escuchar cualquier problema y buscar la solución adecuada.